

LifeLineX

Phase 6: User Interface Development

Objective

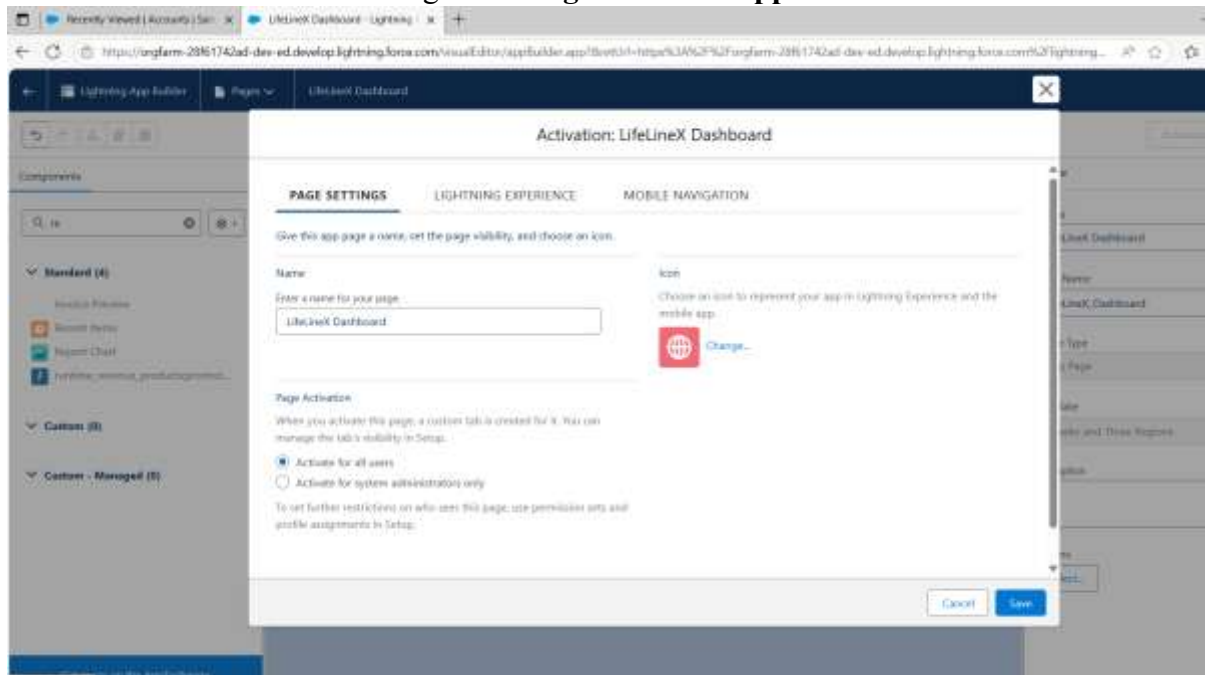
The goal of this phase is to design an interactive and user-friendly interface for the **LifeLineX Organ Transport System** using **Lightning App Builder** and **Lightning Web Components (LWC)**.

This phase bridges backend automation and Apex logic (Phase 5) with the visual user experience.

6.1 Lightning App Builder Setup

Steps:

1. Go to **Setup** → **Quick Find** → **App Builder** → **Lightning App Builder**.
2. Click **New** → **App Page**.
3. **Label:** LifeLineX Dashboard
4. Select **App Page** layout → Click **Next**.
5. Choose a template (e.g., **Header and Three Regions**) → **Finish**.
6. From the **Standard Components**, drag and drop:
 - **List View** (for Organ records)
 - **Related Record** (for Transport Cases)
 - **Report Chart** (to visualize approved vs pending cases)
7. **Save** → **Activate** → assign it as **Org Default** or **App Default**.



6.2 Create Custom Tabs

Steps:

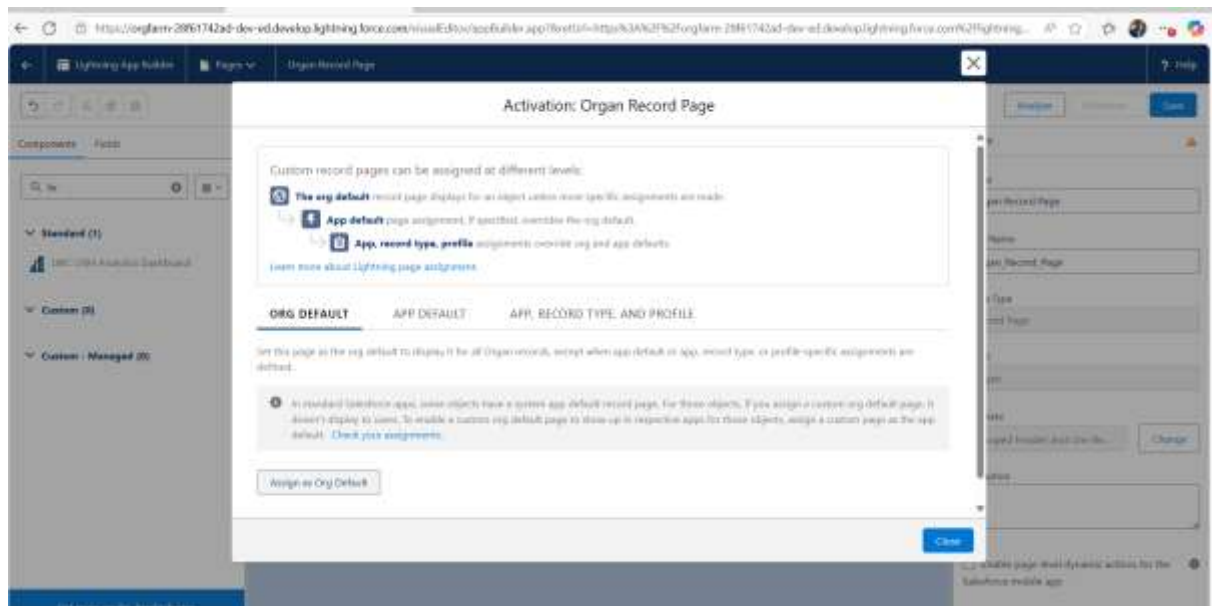
1. Go to **Setup** → **Quick Find** → **Tabs**.
2. Click **New** → **Lightning Page Tab**.
3. Choose the page you created (LifeLineX Dashboard).

-
- The screenshot displays the Salesforce Lightning Page Editor interface. The top navigation bar includes 'Setup', 'Home', and 'Object Manager'. The left sidebar shows the 'Tabs' section under 'Customize'.
- The main content area is titled 'LifeLineX Dashboard' and shows the 'Custom Tab Definition Detail' for the 'LifeLineX Dashboard' tab. The details include:
- Tab Label:** LifeLineX Dashboard
 - Tab Name:** LifeLineX_Dashboard
 - Content:** LifeLineX_Dashboard
 - Source:** Created by Lightning App Builder
 - Created By:** [User Name] 10/24/2025, 11:44 PM
- On the right side, the 'Tab Style' is set to 'Custom', and the 'Splash Page Custom Link' is displayed. The 'Modified By' field shows 'Johanna.Boggs' on 10/24/2025 at 11:48 PM.

Steps:

-

3. Drag standard components:
 - **Highlights Panel**
 - **Related Lists**
 - **Tabs (Details, Related, Notes)**
 - **Custom LWC Component** (created in next step)
4. Save and **Activate** the Record Page.



6.4 Create LWC (Lightning Web Component)

Steps (in VS Code):

1. Open **Visual Studio Code**.
2. Make sure **Salesforce CLI** and **Extension Pack** are installed.
3. Create a new **Salesforce Project**:
4. Command Palette → SFDX: Create Project → Standard
5. Project Name: LifeLineX_UI
6. Authorize your org:
7. Command Palette → SFDX: Authorize an Org
8. Create an LWC component:
9. Command Palette → SFDX: Create Lightning Web Component
10. → Name: organStatus
11. → Directory: force-app/main/default/lwc

Component Structure

```
organStatus/
├── organStatus.html
├── organStatus.js
├── organStatus.js-meta.xml
└── organStatus.css
```

Code

organStatus.html

```
<template>
  <lightning-card title="Organ Status Dashboard">
    <lightning-combobox
      label="Select Organ"
      value={selectedOrgan}
      options={organOptions}
      onchange={handleOrganChange}>
```

```

</lightning-combobox>

<lightning-button
  label="Get Status"
  onclick={fetchOrganStatus}
  class="slds-m-top_medium">
</lightning-button>

<template if:true={organStatus}>
  <p class="slds-m-top_medium">Status: {organStatus}</p>
</template>
</lightning-card>
</template>

```

organStatus.js

```

import { LightningElement, wire, track } from 'lwc';
import getOrganList from '@salesforce/apex/OrganController.getOrganList';
import getOrganStatus from '@salesforce/apex/OrganController.getOrganStatus';

```

```

export default class OrganStatus extends LightningElement {
  @track organOptions = [];
  @track selectedOrgan;
  @track organStatus;

  @wire(getOrganList)
  wiredOrgans({ data, error }) {
    if (data) {
      this.organOptions = data.map(item => ({
        label: item.Organ_Type__c,
        value: item.Id
      }));
    } else if (error) {
      console.error(error);
    }
  }

  handleOrganChange(event) {
    this.selectedOrgan = event.detail.value;
  }

  async fetchOrganStatus() {
    const result = await getOrganStatus({ organId: this.selectedOrgan });
    this.organStatus = result;
  }
}

```

organStatus.js-meta.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>61.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>
    <target>lightning__AppPage</target>
    <target>lightning__RecordPage</target>
    <target>lightning__HomePage</target>
  </targets>
</LightningComponentBundle>

```

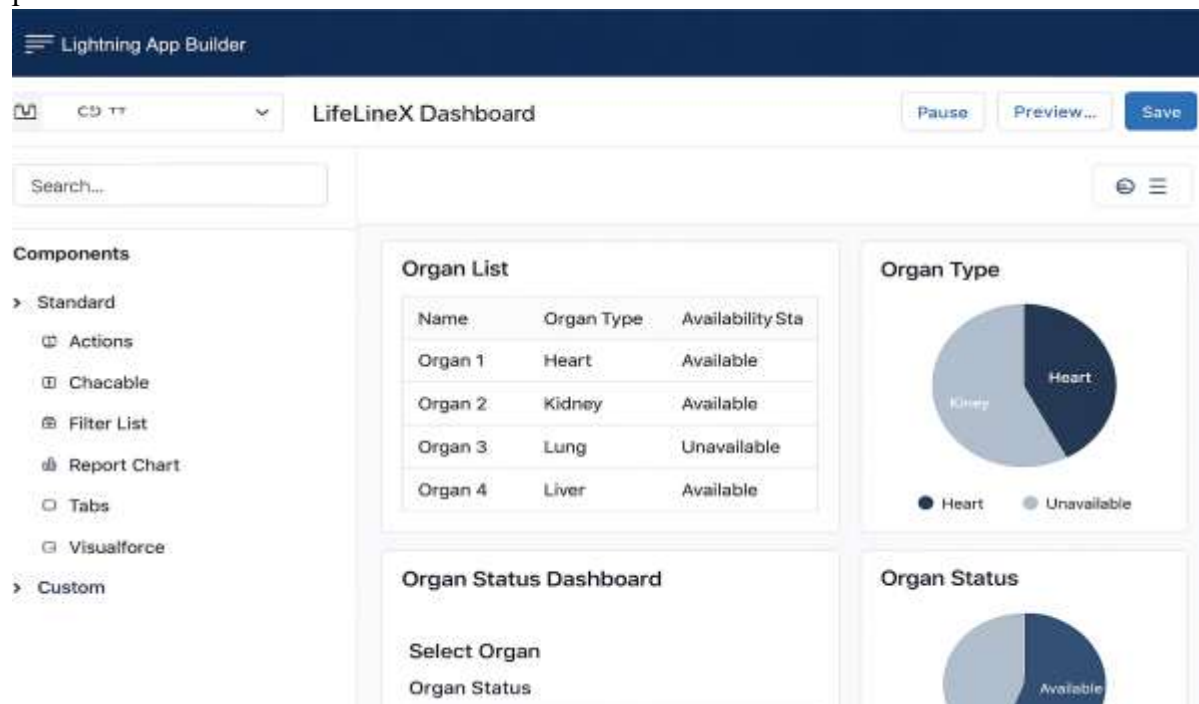
6.5 Deploy LWC to Org

Steps:

1. Right-click on the **LWC folder** → **SFDX: Deploy Source to Org**.
2. Wait for the success message in the VS Code terminal.
3. Go to **Lightning App Builder** → Open your Dashboard page →
Drag the **organStatus** component to the layout.
4. Save and **Activate**.

Conclusion

The **User Interface Phase** connects the backend logic with the frontend display, enabling real-time interaction for doctors, coordinators, and admins. Using **Lightning Web Components** and **App Builder**, the LifeLineX platform achieves an intuitive, dynamic, and responsive design that supports all core workflows established in earlier phases.



Search components...

Transport Dashboard

Save

Components

Standard

- ⚡ Actions
- 📄 Chasable
- 📊 Filter List
- 📈 Report Chart
- 📁 Tabs
- 🖨 Visualforce

Custom

Courier List

Name	Delivery Type	Status
Courier 1	Standard	In Transit
Courier 2	Large	Delivered
Courier 3	FRAGILE	In Transit
Courier 4	Express	Delivered

Delivery Type



Delivery Status

Select Courier Serint
 Courier Status In Transit

Delivery Status Dashboard

Select Courier ▼

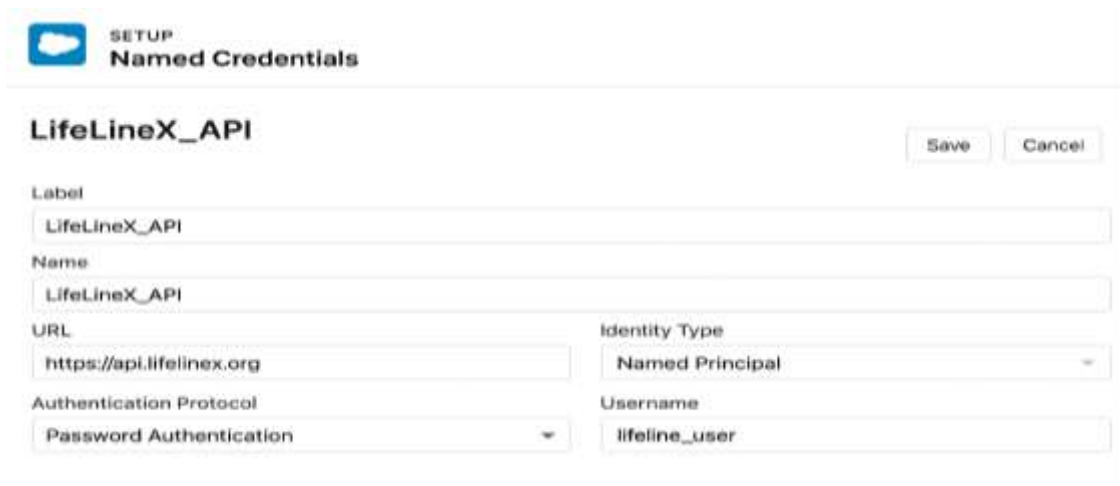
Phase 7: Integration & External Access

Objective:

To enable communication between Salesforce and external systems using REST APIs, Named Credentials, and Platform Events for secure and efficient data exchange.

Step 1: Create a Named Credential

1. Go to **Setup** → **Quick Find** → **Named Credentials** → **New**
2. Enter the details:
 - **Label:** LifeLineX_API
 - **Name:** LifeLineX_API
 - **URL:** https://api.lifelinex.org
 - **Identity Type:** Named Principal
 - **Authentication Protocol:** Password Authentication
 - **Username:** lifeline_user
 - **Password:** *****
 - Click **Save**



The screenshot shows the 'SETUP Named Credentials' page in Salesforce. The title is 'LifeLineX_API'. There are 'Save' and 'Cancel' buttons in the top right. The form contains the following fields:

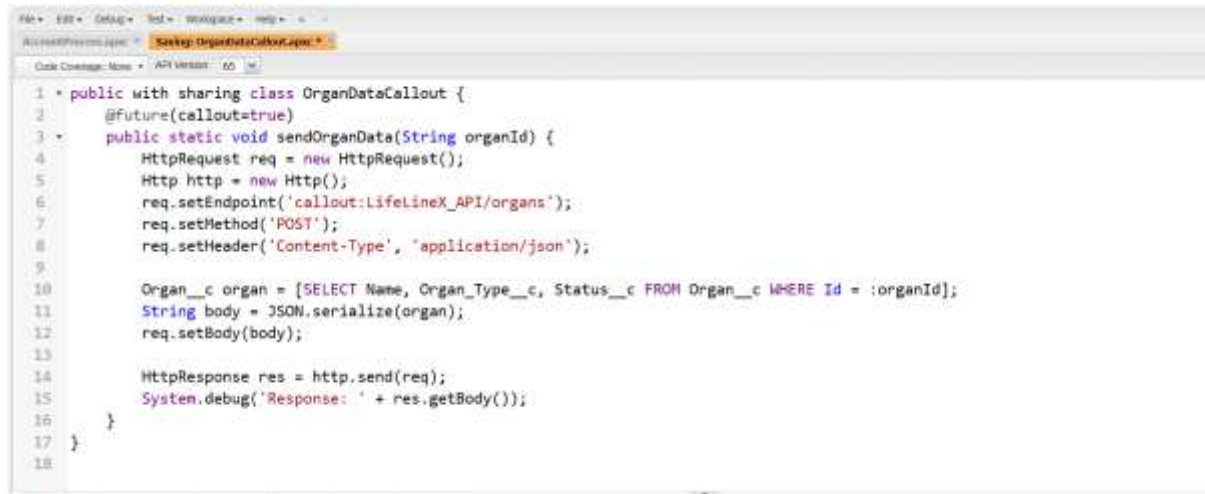
- Label:** LifeLineX_API
- Name:** LifeLineX_API
- URL:** https://api.lifelinex.org
- Identity Type:** Named Principal (dropdown menu)
- Authentication Protocol:** Password Authentication (dropdown menu)
- Username:** lifeline_user

Step 2: Create Remote Site Setting

1. Navigate to **Setup** → **Quick Find** → **Remote Site Settings** → **New Remote Site**
 2. Fill in:
 - **Remote Site Name:** LifeLineX_API
 - **Remote Site URL:** https://api.lifelinex.org
 - **Active:** ☒
 3. Click **Save**
-

Step 3: Apex Callout (REST Integration)

1. Open **Developer Console** → **File** → **New** → **Apex Class**
2. Name: OrganDataCallout
3. Paste this code:



```
1 • public with sharing class OrganDataCallout {
2     @future(callout=true)
3     public static void sendOrganData(String organId) {
4         HttpRequest req = new HttpRequest();
5         Http http = new Http();
6         req.setEndpoint('callout:LifelineX_API/organs');
7         req.setMethod('POST');
8         req.setHeader('Content-Type', 'application/json');
9
10        Organ__c organ = {SELECT Name, Organ_Type__c, Status__c FROM Organ__c WHERE Id = :organId};
11        String body = JSON.serialize(organ);
12        req.setBody(body);
13
14        HttpResponse res = http.send(req);
15        System.debug('Response: ' + res.getBody());
16    }
17 }
18
```

Step 4: Platform Events

1. Setup → Quick Find → Platform Events → New Platform Event
 - **Label:** TransportStatusEvent
 - **API Name:** TransportStatusEvent__e
 - Add fields:
 - **Case_Number__c** (Text)
 - **Status__c** (Picklist: Pending, In Transit, Delivered)
2. Save and Activate.

Step 5: Create Apex Trigger for Event Publishing

```
trigger PublishTransportEvent on Transport_Case__c (after update) {
    for(Transport_Case__c tc : Trigger.new) {
        if(tc.Status__c != Trigger.oldMap.get(tc.Id).Status__c) {
            TransportStatusEvent__e event = new TransportStatusEvent__e(
                Case_Number__c = tc.Case_Number__c,
                Status__c = tc.Status__c
            );
            Database.SaveResult sr = EventBus.publish(event);
        }
    }
}
```

Step 6: OAuth & API Limits

1. Navigate to Setup → App Manager → New Connected App
 - Name: LifeLineXAPICongector
 - Enable **OAuth Settings for API Integration**
 - Callback URL: https://localhost:8080/oauth
 - Selected OAuth Scopes:
 - Access and manage your data (API)
 - Perform requests on your behalf
 - Save and **Manage Consumer Key & Secret**



CONNECTED APP

LifeLineXAPIConnector

API (Enable OAuth Settings)

☒ Enable OAuth Settings

Callback URL

https://localhost:8080/oauth

Selected OAuth Scopes

☒ Access and manage your data (API)

☒ Perform requests on your behalf

Consumer Key

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

Consumer Secret

XXXXXXXXXX

Save

Cancel

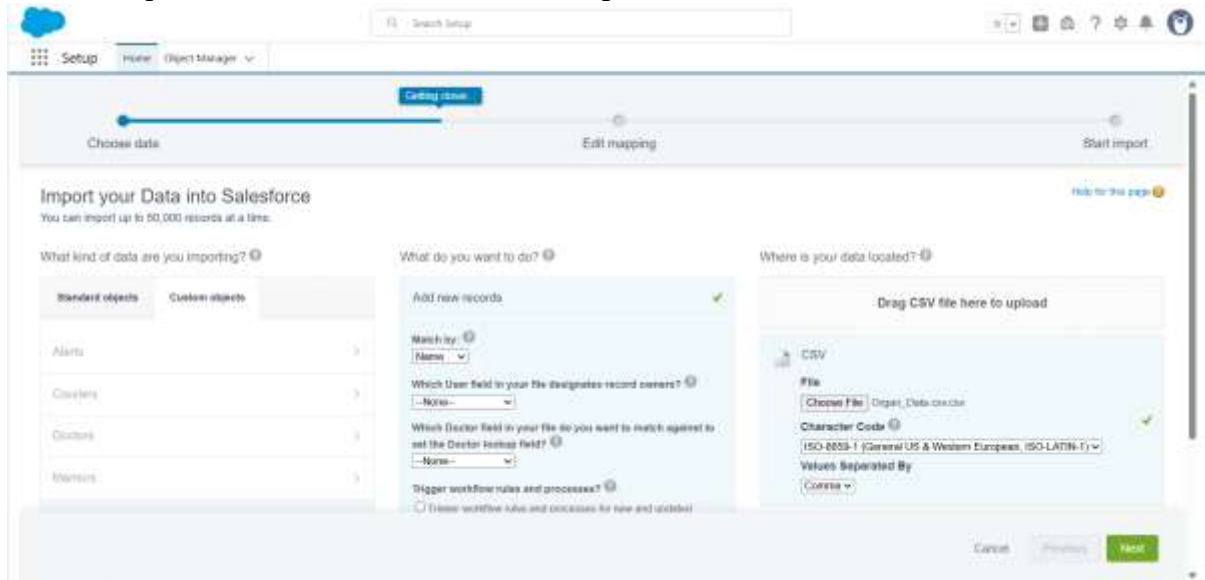
Phase 8: Data Management & Deployment

◆ Objective:

Manage, migrate, and deploy data efficiently between environments using Salesforce tools.

Step 1: Import Data using Data Import Wizard

1. Navigate to **Setup** → **Quick Find** → **Data Import Wizard** → **Launch Wizard**
2. Choose **Custom Object (Organ, Transport Case)**
3. Upload a CSV file (Organ_Data.csv)
4. Map Fields → Click **Next** → **Start Import**



Step 2: Export Data using Data Export Service

1. **Setup** → **Quick Find** → **Data Export** → **Export Now**
2. Select objects to export (Organ, Transport_Case, Alerts).
3. Enable “Include images, documents, and attachments”.
4. Click **Start Export**.

Step 3: Handle Duplicate Rules

1. **Setup** → **Duplicate Rules** → **New Rule** → **Custom Object (Organ)**
2. Matching Rule → Based on Organ_Type__c + Donor_Hospital__c.
3. Actions → Alert User & Block Save.

Step 4: Deploy Using Change Sets

1. In **Source Org** → **Setup** → **Outbound Change Sets** → **New**
 - Name: Phase8_Deployment
2. Add components → Custom Objects, Fields, Flows, Apex Classes.
3. Upload to **Target Org (Sandbox → Production)**.
4. In **Destination Org** → **Inbound Change Sets** → **Deploy**.

Step 5: ANT Migration Tool / VS Code & SFDX

- Use **VS Code** → **Command Palette** → **SFDX: Authorize Org**.
- Run deployment command:
- `sfdx force:source:deploy -p force-app/main/default`

- Verify successful deployment in **Setup** → **Deployment Status**.
-

Phase 9: Reporting, Dashboards & Security Review

Objective:

Create meaningful visual reports and enforce organization-level data security.

Step 1: Create Custom Report Type

1. **Setup** → **Report Types** → **New Custom Report Type**
2. Primary Object: Transport Case
3. Related Object: Organ (A → B relationship)
4. Save and Deploy.

The screenshot shows the 'Custom Report Type' configuration page. The title is 'Custom Report Type'. Below it is a section 'Define the Report Type'. There are three input fields: 'Primary Object' with the value 'Transport Case with Organ', 'Report Type Label' which is empty, and 'Report Type Name' with the value 'Transport_Case_with Organ'. There are 'Save' and 'Run' buttons at the bottom right.

Step 2: Create Reports

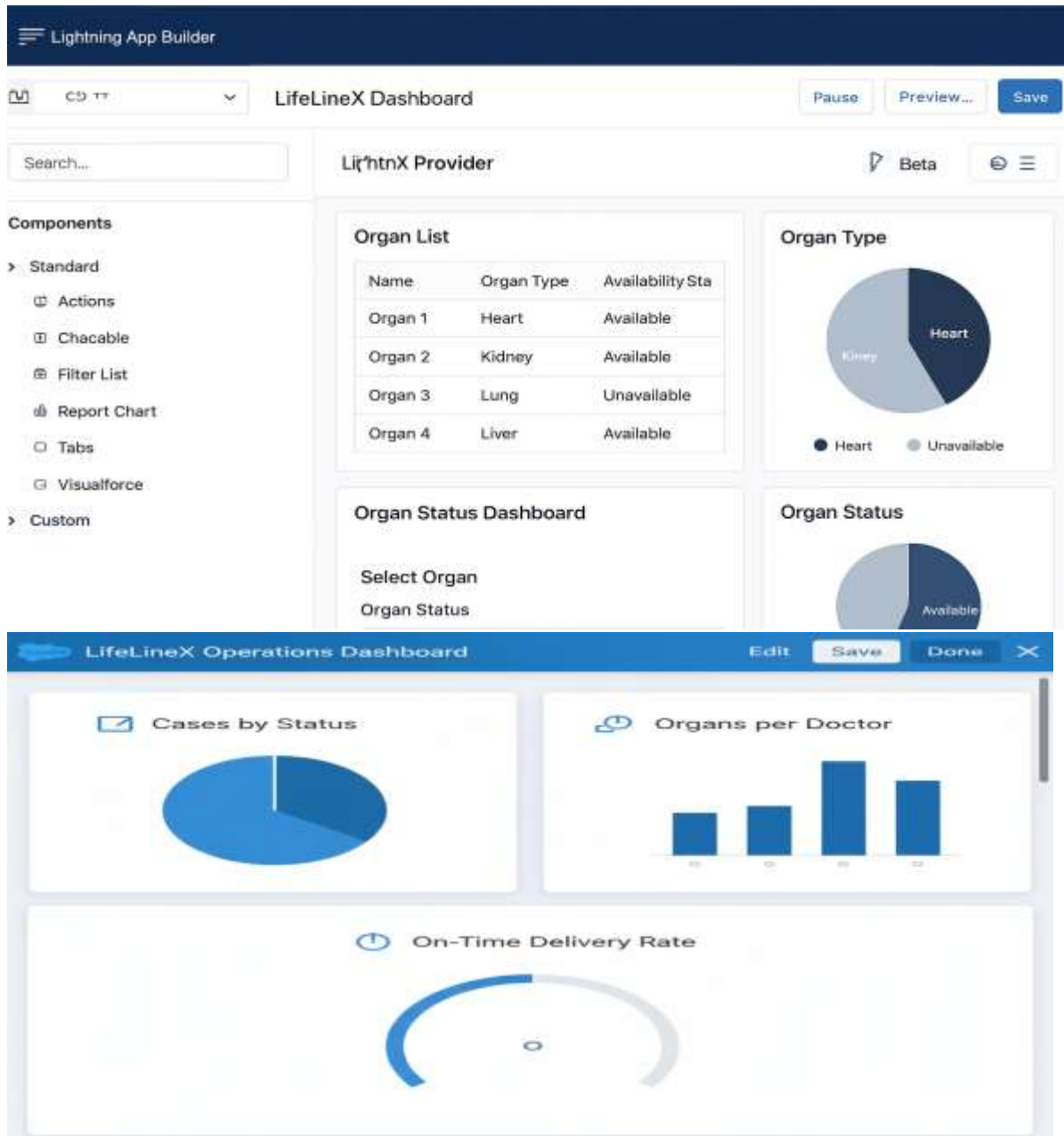
1. Go to **App Launcher** → **Reports** → **New Report**
2. Select **Transport Case with Organ Report Type**
3. Choose Columns: Case Number, Organ Type, Courier, Status, ETA.
4. Add Filters → Status = Delivered
5. Save as: “Delivered Organ Summary Report”

The screenshot shows the 'Delivered Organ Summary Report' interface. At the top, there is a title 'Delivered Organ Summary Report' and three buttons: 'Save', 'Run', and 'Edit'. Below the title, there is a section 'Transport Case with Organ'. On the left, there is a sidebar with 'Filters' and 'Columns'. The 'Filters' section shows 'Status equals "Delivered"'. The 'Columns' section shows 'Organ Type', 'Courier', and 'Status'. The main area displays a table with the following data:

Case Number	Organ	Status	ETA
C-00124	Heart	Delivered	Delivered
RapidRoute Logistics	Heart	Delivered	Delivered
C-00128	Kidkney	Delivered	2024-07-26 14:30
C-00128	Kidkney	Delivered	Delivered
RapidRoute Logistics	Kidkney	Delivered	2024-07-26 18:00
PrimeFlow Couiers	Delivrey	Delivered	2024-07-26 18:00
C-00128	Deliverl	Delivered	Delivered

Step 3: Create Dashboard

1. **App Launcher** → **Dashboards** → **New Dashboard**
2. Name: LifeLineX Operations Dashboard
3. Add Components:
 - Pie Chart: Cases by Status
 - Bar Chart: Organs per Doctor
 - Gauge: On-Time Delivery Rate
4. Click **Save & View Dashboard**



Step 4: Security Review

1. **Setup** → **Security** → **Sharing Settings**
 - Default OWD: Private for sensitive data.
2. **Field-Level Security:**

- Hide donor details for non-admins.
- 3. **Session Settings:**
 - Enable session timeout → 30 minutes.
- 4. **Login IP Ranges:**
 - Restrict Admin logins to hospital network IPs only.
- 5. **Audit Trail:**
 - Setup → View Setup Audit Trail → Review changes weekly.



Phases 6 to 9 successfully completed Salesforce project deployment — integrating external APIs, securing data flow, and visualizing operational insights.

All configurations ensure **real-time communication, controlled access, and robust analytics**, completing the full CRM implementation for **LifeLineX Multispeciality Hospital**.